arXiv:2503.21782v2 [cs.CV] 19 Mar 2026

Mobile-VideoGPT: Fast and Accurate Model for Mobile Video Understanding

Abdelrahman Shaker[1,*], Muhammad Maaz[1], Chenhui Gou[2], Hamid Rezatofighi[2], Salman Khan[1], and Fahad Shahbaz Khan[1,3]

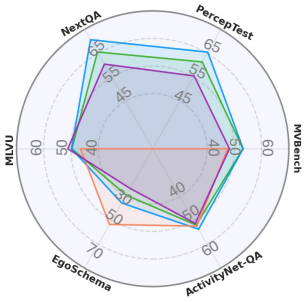
[1]Mohamed Bin Zayed University of Artificial Intelligence
[2]Monash University [3]Linköping University

[*]Corresponding author: abdelrahman.youssief@mbzuai.ac.ae

Abstract. Video understanding models often struggle with high computational requirements, extensive parameter counts, and slow inference speed, making them inefficient for practical use on edge devices. To tackle these challenges, we propose Mobile-VideoGPT, an efficient multimodal framework designed to operate with fewer than a billion parameters. Unlike traditional video large multimodal models (LMMs), Mobile-VideoGPT consists of lightweight dual visual encoders, efficient projectors, and a small language model (SLM), enabling real-time inference on resource-constrained platforms. To further improve efficiency, we present an Attention-Based Frame-Scoring mechanism to select the keyframes, along with an efficient token projector that prunes redundant visual tokens and preserves essential contextual cues. We evaluate our model across well-established six video understanding benchmarks (e.g., MVBench, EgoSchema, NextQA, and PerceptionTest), and our results demonstrate that Mobile-VideoGPT-0.5B achieves up to 7.3 tokens per second on NVIDIA Jetson Orin edge devices and 45.9 tokens per second on a single NVIDIA RTX A6000. Despite its compact size, it outperforms existing state-of-the-art 0.5B-parameter models by an average of 6 points, while using 40% fewer parameters and delivering more than 2× higher throughput across both edge and desktop platforms. Our code and models are publicly available at: https://github.com/Amshaker/Mobile-VideoGPT.

1 Introduction

Multimodal Large Language Models (MLLMs) have emerged as a pivotal area of research and demonstrated exceptional abilities in video understanding tasks, including video captioning, visual question-answering, and visual grounding [32,38,21]. Remarkable performance on these tasks can be witnessed in proprietary models such as GPT-4o [45] and Gemini-1.5-Pro [10] or open-source alternatives such as Video-LLaVA [32] and Video-ChatGPT [38]. Despite the promising performance of these methods across specific benchmarks, deploying these models on edge devices presents a significant challenge.



Model	Total Params (Billion) ↓	Model Size (GB) ↓	Throughput (Tokens/second)	
			Jetson Orin Nano ↑	RTX A6000 ↑
Mobile-VideoGPT-0.5B	**0.6**	**2.0**	**7.3**	**45.9**
Mobile-VideoGPT-1.5B	1.6	4.1	6.1	41.0
LLaVA-one-vision-0.5B	1.0	5.9	3.4	22.7
LLaVA-Mini-8B	8.4	20.8	NA	7.6

Fig. 1. Left: Performance comparison of Mobile-VideoGPT against state-of-the-art efficient video understanding models across six benchmarks (PercepTest, NextQA, MVBench, EgoSchema, ActivityNet-QA, and MLVU). Mobile-VideoGPT achieves consistently strong accuracy across diverse video reasoning tasks. **Right**: Efficiency comparison in terms of total parameters, model size, and inference throughput (tokens/second) on Jetson Orin Nano and RTX A6000. Despite using significantly fewer parameters and a smaller memory footprint, Mobile-VideoGPT has higher throughput with better accuracy–efficiency trade-off and strong suitability for edge devices.

The design of large multimodal architectures often results in a substantial number of parameters and high computational demands, rendering them impractical for resource-constrained environments. For instance, the recently introduced LLaVa-Mini [65] proposes an image and video MLLM based on a single one-vision token, yet its total number of parameters is 8.4B, and the model size is more than 20GB, making it impractical for real-time performance and edge deployment. Hence, there is a need for efficient video understanding models and training paradigms that are capable of retaining strong performance without exceeding the resource limitations.

Over the past decade, techniques such as knowledge distillation [16,51], model pruning [14,15], efficient attention [52,40], and parameter-efficient fine-tuning [17] have partially addressed the challenge of reducing model complexity under resource constraints. However, these strategies come with notable performance trade-offs or require specialized hardware accelerators. Moreover, while they may effectively reduce the size of individual components (e.g., visual or text encoders), ensuring efficient projection and integration across modalities for video understanding and with advanced capabilities remains a challenge.

To this end, we introduce Mobile-VideoGPT, an efficient video understanding language model that balances between robust performance and high throughput on edge devices (i.e, Jetson Orin Nano [42]). We demonstrate the efficiency and the effectiveness of Mobile-VideoGPT across multiple video conversation benchmarks, including ActivityNet-QA, EgoSchema, MLVU, MVBench, NextQA, and PerceptionTest, where it not only is faster than the previous SoTA, but also outperforms them (See Fig. 1). Our contributions in this work are as follows:

1. A lightweight multimodal model designed specifically for real-time video understanding with a focus on edge device deployment.
2. We propose a novel frame scoring and selection strategy that optimally selects the representative video key-frames, along with efficient token projection, to enhance the model's ability to maintain visual fidelity while reducing computational overhead for efficient processing.
3. Our architecture is carefully optimized for edge deployment, featuring a compact model size of just 1GB in FP16 precision and requiring only 3GB of VRAM. Owing to its efficient design, it achieves a high throughput of 7.3 tokens per second on Jetson Orin Nano [42].
4. Mobile-VideoGPT showcases superior performance compared to competitive models across six video understanding benchmarks, achieving an average improvement of 6 points over LLaVA-OneVision-0.5B [21]. Our model is also 2× faster, while being less than 3× its size.

2 Related Work

Small Language Models (SLMs): In the past year, the rise of large language models (LLMs) has driven significant progress in natural language processing, including proprietary models like GPT-4 [44], and open-source models such as LLaMA [55] and DeepSeek [34]. Although these models have sparked increased interest in enhancing the performance of LLMs, the large size of these models often leads to substantial memory consumption and slow processing speeds. Moreover, the energy demands associated with training and operating these models raise important sustainability issues, which are becoming increasingly urgent as LLMs expand in scale and complexity. To address this, there has been a growing trend in the development of small and medium-sized LLMs, such as Qwen2-0.5B [59] and Llama3.2-1B [1], specifically designed for resource-constrained environments. Given the limited computational capabilities of edge devices such as mobile phones, autonomous vehicles, and embodied AI systems, several lightweight LLMs [30,63,37,7] have gained significant attention. TinyLLaMA [63] conducts great open-source work on their 1.1B base and chat models with the same architecture and tokenizer as LLaMa2 [54]. MobileLLaMA [7] scales down the architecture of LLaMA and releases 1B and 3B models trained from scratch with public datasets. Recently, MobileLLM [37] optimizes sub-billion parameter models by proposing an immediate block-wise weight-sharing approach. These compact models are optimized to deliver efficient performance while addressing the unique constraints of resource-limited environments. Another vital line of research involves applying model compression techniques for LLMs, such as pruning [9], quantization [57,27], and low-rank decomposition [60]. These techniques reduce the computational demands of LLMs and enhance their practicality for deployment on resource-constrained devices.

Vision Language Models: The advancement of visual language models (VLMs) has been significantly shaped by the introduction of CLIP [48], which utilizes a contrastive loss to align vision and language embeddings within a unified

representation space. Recent progress in LLMs has further enhanced the development of VLMs, as demonstrated in Kosmos-2 [47], GLaMM [49], BLIP-2 [22], MiniGPT-4 [68], LLaVA [36] and PALO [50]. These models combine visual and linguistic features, leveraging advanced cross-modal designs, such as cross-attention mechanisms, Q-Former modules, or simple multi-layer perceptron layers.

Video Language Models: Current video understanding models typically rely on image encoders [23,2,32], video encoders [39,24], or dual spatial/temporal vision stacks [8,6] to process visual inputs alongside advanced LLMs. Image encoders capture rich spatial details from frame sequences but often miss explicit temporal cues essential for complex action recognition. Video encoders provide temporal context but are constrained by efficiency, typically processing sparse frames at lower resolutions and thus losing both contextual and spatial fidelity. VideoMamba [25], a recent parameter-efficient video encoder, addresses these issues by extending the Mamba architecture to the video domain, combining linear-complexity operators for efficient long-term modeling with strong scalability.

Goldfish [2] and VideoGPT+ [39] improve scalability by segmenting videos into short clips processed independently. LLAVA-OneVision [21] advances this direction by introducing a unified framework for both image and video tasks, using a single vision encoder for frame-level feature extraction while relying on the language model's positional embeddings for temporal understanding. Despite these advances, most approaches remain impractical for edge deployment due to high computational demands, large memory footprints, and reliance on powerful accelerators. Their complex architectures with large encoders and high-resolution inputs require substantial GPU and storage resources, limiting their use in real-time, resource-constrained environments.

More recently, attention-style frame selection [31,53,4] and efficient projection layers to reduce visual tokens [28,56,19] have emerged to mitigate these challenges. These limitations also motivate our work to develop a lightweight, resource-efficient video understanding model dedicated to edge devices by reducing the model size and memory usage and enabling real-time inference without compromising performance.

3 Mobile-VideoGPT

We introduce Mobile-VideoGPT, specifically designed for efficient video understanding. It incorporates both an image encoder and a video encoder; each optimized for speed and number of parameters, along with compact, efficient projection modules that project the latent visual features into a suitable format for small language model.

3.1 Overall Architectural Design

Inspired by recent work [39,69], Mobile-VideoGPT adopts a dual-encoder design combined with a small language model (SLM). As illustrated in Fig. 2, the

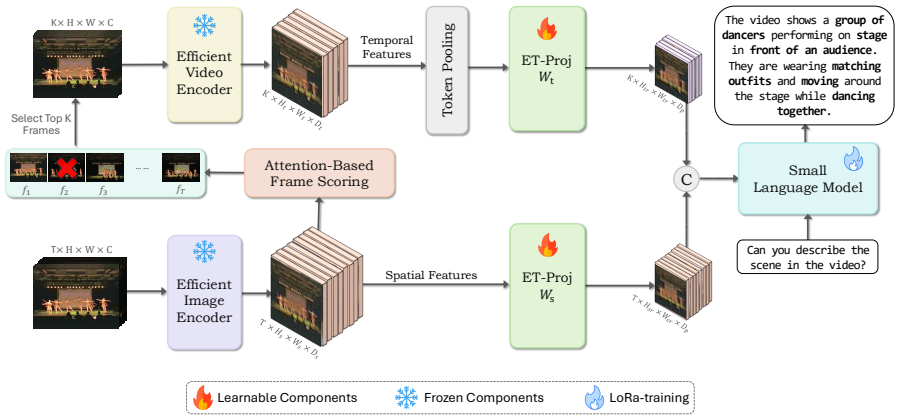


Fig. 2. Overview of the proposed Mobile-VideoGPT. The pipeline begins by extracting spatial features from all video frames with an efficient image encoder. These features then flow into an attention-based frame scoring mechanism, which identifies the top salient K frames. Next, an efficient video encoder processes these selected frames to capture temporal dynamics. The resulting spatial and temporal representations are projected into a unified vision–language space using an efficient token projector (ET-Proj). Finally, the projected tokens are fused, and a small language model leverages these tokens to generate comprehensive responses to video-based questions.

proposed framework consists of: (1) a CLIP-based image encoder [48] for capturing spatial and semantic features, (2) a lightweight VideoMamba encoder [25] for extracting temporal information, (3) an Efficient Token Projection (ET_{Proj}) module for feature reduction and projection, and (4) an SLM for generating context-rich textual responses.

We start by uniformly sampling T frames from the input video. For each frame $t \in \{1, \ldots, T\}$, we extract spatial embeddings via the CLIP-based image encoder, forming

$$\mathbf{F}_{\text{clip}} \in \mathbb{R}^{T \times H_s \times W_s \times D_s},$$

where H_s and W_s denote the spatial dimensions for image features, and D_s the spatial feature depth. Then, we apply attention-based frame scoring (Section 3.2) to select the top-K key-frames, which are passed to the VideoMamba encoder-M to encode the temporal features. This yields

$$\mathbf{F}_{\text{vmamba}} \in \mathbb{R}^{K \times H_t \times W_t \times D_t},$$

where H_t and W_t represent spatial dimensions for the video features, and D_t is the temporal feature depth.

Next, both the clip-based and the VideoMamba embeddings are processed through Efficient Token Projection modules. Concretely, ET_{Proj} reduces and transforms

each modality into a unified vision–language embedding space:

$$\widehat{\mathbf{F}}_{\text{clip}} = \text{ET}_{\text{Proj}_s}\big(\mathbf{F}_{\text{clip}}\big) \;=\; \mathbf{F}_{\text{clip}}\,\mathbf{W}_s, \tag{1}$$

$$\widehat{\mathbf{F}}_{\text{vmamba}} = \text{ET}_{\text{Proj}_t}\big(\mathbf{F}_{\text{VMamba}}\big) \;=\; \mathbf{F}_{\text{VMamba}}\,\mathbf{W}_t, \tag{2}$$

where $\mathbf{W}_s$ and $\mathbf{W}_t$ are learnable weight matrices for spatial and temporal features, respectively. We then fuse $\widehat{\mathbf{F}}_{\text{clip}}$ and $\widehat{\mathbf{F}}_{\text{vmamba}}$ into a single token sequence, which a *small language model (SLM)* consumes to produce context-rich responses to video-based queries.

3.2 Frame Selection

A straightforward approach would be to feed all T sampled frames directly into the video encoder. However, this often proves redundant and inefficient. Therefore, we propose an *attention-based frame scoring* mechanism to select only the most salient K frames ($K < T$), substantially reducing the computational load. In our setup, $K = T/2$. Let $\mathbf{F}_{\text{clip}} \in \mathbb{R}^{T\times H_s\times W_s\times D_s}$ be the image features from the CLIP-based encoder. We flatten all T frames into $\mathbf{F}_{\text{sp}} \in \mathbb{R}^{S\times D_f}$, where $S = T \times H_I \times W_I$ and D_f is the spatial feature dimension. We then compute a *spatial attention* matrix:

$$\text{SA} \;=\; \text{Softmax}\Big[\frac{\mathbf{F}_{\text{sp}}\,\mathbf{F}_{\text{sp}}^{\top}}{\sqrt{d_f}}\Big], \tag{3}$$

where $\mathbf{F}_{\text{sp}}$ are the flattened spatial features. Next, we sum the attention scores each token receives to produce a frame-wise importance vector $T_{\text{score}} \in \mathbb{R}^T$. We then select the top-K frames via:

$$\text{KF} \;=\; \text{Top-K}\big(T_{\text{score}},\, K\big). \tag{4}$$

Finally, we feed the selected key-frames (KF), after reshaping, into VideoMamba to capture temporal dynamics, yielding $\mathbf{F}_{\text{vmamba}} \in \mathbb{R}^{K\times H_t\times W_t\times D_t}$. In our experiments, we set $T = 16$ and $K = 8$. By discarding low-importance frames, this approach effectively reduces redundant computations while preserving crucial contextual information.

3.3 Efficient Token Projection

Fig. 3 (d) describes the *Efficient Token Projector* (ET_{Proj}), which compresses and re-encodes tokenized image/video features while preserving crucial spatiotemporal information. The module comprises three stages: (1) Feed-forward network (FFN) that transforms the input features $\mathbf{x} \in \mathbb{R}^{B\times N\times C_{\text{in}}}$ into $\mathbb{R}^{B\times N\times C_{\text{out}}}$, where B is the batch size, N is the visual tokens, and C is the hidden dimension. Then, we reshape the projected tensor into $\mathbf{X} \in \mathbb{R}^{B\times C_{\text{out}}\times H\times W}$. (2) Token reduction with AdaptivePool to reduce the spatial size (H, W) to a smaller (H_r, W_r), and (3) Convolution-based positional encoder with skip connection to enhance

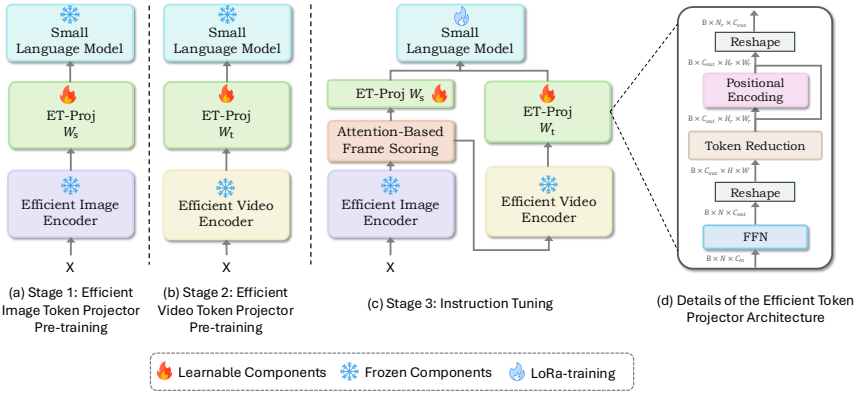


Fig. 3. Overview of Mobile-VideoGPT training strategy and the architecture of the Efficient Token Projector (ET-Proj). (a–c) Training proceeds in three stages: pre-training the image token projector with an efficient image encoder, extending to video projector pre-training with a video encoder, and finally instruction tuning where both projectors are learnable and LoRA fine-tuning is applied to the language model. (d) ET-Proj uses a lightweight feedforward network (FFN) to refine token embeddings, a token reduction step via global average pooling, and a positional encoding module with skip connection to retain spatial and temporal context.

the key positional dependencies via learned offsets. Formally, the block can be summarized as:

$$\mathbf{X}'' = \mathrm{PosEnc}\Big(\mathrm{TokenReduction}\big(\mathrm{FFN}(\mathbf{x})\big)\Big). \tag{5}$$

By reducing the spatial tokens while retaining global features and positional cues, our projector provides an efficient mechanism for compressing high-dimensional tokens in both images and videos, substantially reducing computational overhead while preserving spatiotemporal fidelity.

4 Experiments

4.1 Evaluation Benchmarks

We conduct a quantitative evaluation of Mobile-VideoGPT across ***six well-established benchmarks***, namely: i) ActivityNet-QA [62], ii) EgoSchema [41], iii) MLVU [67], iv) MVBench [26], v) NExTQA [58], and vi) PerceptionTest [46]. These benchmarks collectively cover diverse tasks such as multiple-choice question answering, open-ended generation, spatiotemporal reasoning, and detailed action understanding, offering a comprehensive evaluation of our model's capabilities across real-world and research-oriented scenarios. Specifically, **ActivityNet-QA** [62] evaluates activity and action understanding on a test set of 8K samples derived from the ActivityNet dataset, using GPT-3.5-Turbo for open-ended

evaluation following [38]. **EgoSchema** [41] targets egocentric video understanding through 5K MCQ samples sourced from Ego4D [13], with results evaluated via an online server. **MLVU** [67] probes long video comprehension across nine distinct tasks spanning both MCQ and open-ended generation, and we report results on the 2,593-sample dev set following [21]. **MVBench** [26] offers a broad assessment of video understanding in the wild through 20 fine-grained sub-tasks, ranging from action sequence and prediction to egocentric navigation and counterfactual inference—totaling 4K MCQs. **NExTQA** [58] focuses on temporal action understanding, evaluating causal and temporal action reasoning as well as scene comprehension on an MCQ test set of 8,564 samples. Finally, **PerceptionTest** [46] assesses perception and reasoning capabilities on a validation set of 19,140 samples, ensuring a thorough evaluation of our model's real-world applicability. More details are provided in the supplementary material.

4.2 Implementation Details

In our experiments, we utilize CLIP-B/16 [48] as the image encoder, and for the video encoder, we use VideoMamba-M [25], alongside the Qwen-2.5, 0.5/1.5B [59] LLM. Both image and video encoders process inputs at a resolution of 224×224. Our training pipeline consists of two warm-up phases followed by an instruction-tuning phase. During warm-up, we separately train with the image and video encoders on the CC-595K dataset [35], optimizing only the vision-to-language adapters while keeping the remaining model parameters frozen. In the instruction-tuning phase, we apply LoRA [18] with a rank of $r = 64$ to the LLM, while the vision-to-language adapters are fully trained and the vision encoders remain frozen. The learning rate is set to $1e^{-3}$ during warm-up and $2e^{-4}$ during instruction tuning. During instruction tuning, we first sample videos at 1 FPS, followed by a uniform sampling of 16 frames. The value of K is set to 8 for attention-based frame scoring.

We use instruction-tuning data from VideoGPT+ [39], which consists of Kinetics-710 [20], Something-Something-v2 [12], conversations from VideoChat [23], CLEVRER [61], the VQA dataset from WebVid [3], and NExT-QA [58], along with the PerceptionTest [46] subset from LLaVA-178K [66] and a subset of VideoChat2-IT [26]. The instruction-tuning dataset amounts to approx. 1M samples, and we train our model for two epochs.

Although we use only a fraction of the instruction-tuning dataset compared to previous SoTA approaches (e.g., LLaVA-OneVision, which utilizes approx. 4.8M samples combining images, multi-image, and videos), our ablations indicate that the observed performance gains are primarily driven by our architectural improvements—specifically, the dual-encoder design, frame-selection strategy, and efficient projection modules. We anticipate that scaling to a larger instruction-tuning dataset would further improve performance, but the current results demonstrate that our efficient MLLM architecture can achieve better results even with less number of samples.

Table 1. Performance comparison of Mobile-VideoGPT against state-of-the-art video understanding models across six benchmarks. We report accuracy (%) alongside efficiency metrics including total parameters, input resolution, frame count, and throughput. Our models achieve competitive performance with significantly higher throughput (frames per second) than larger counterparts. Throughput is evaluated on a single NVIDIA RTX A6000 GPU.

Model	Total Params	Input Res.	Num Frames	Throughput	ActNet-QA	EgoSchema	MLVU	MVBench	NextQA	PercepTest
	↓	↓	↓	↑	test	test	m-avg	test	mc	val
GPT-4V [43]	-	512×512	32	-	57.0	-	49.2	43.5	-	-
Gemini-1.5-Flash [11]	-	768×768	32	-	55.3	65.7	-	-	-	-
Gemini-1.5-Pro [11]	-	768×768	32	-	57.5	72.2	-	-	-	-
>7B Models										
LLaMA-VID-7B [29]	7.3B	336×336	1fps	6.0	47.4	38.5	33.2	41.4	-	-
Video-LLaVA-7B [33]	7.3B	224×224	8	12.5	38.4	45.3	-	43.1	-	-
VideoChat2-7B [23]	7.3B	224×224	16	11.4	49.1	54.4	-	60.4	79.5	47.3
LongVA-7B [64]	7.4B	224×224	8	9.2	50.0	-	56.3	-	68.3	-
LLaVA-Mini-8B [65]	8.4B	336×336	1fps	4.6	52.3	51.2	42.8	44.5	-	-
<7B Models										
InternVL2.5-2B [5]	2.2B	448×448	32	21.8	54.1	-	51.6	57.5	75.6	66.3
Video-ChatGPT-3.8B [38]	4.2B	224×224	100	14.1	35.2	36.2	31.3	32.7	-	-
LLaVA-OneVision-0.5B [21]	1.0B	384×384	32	22.7	50.5	26.8	50.3	47.2	57.2	49.2
Mobile-VideoGPT-0.5B	**0.6B**	**224×224**	**16**	**45.9**	**51.6**	**31.4**	**47.9**	**53.5**	**65.4**	**58.7**
Mobile-VideoGPT-1.5B	**1.6B**	**224×224**	**16**	**41.0**	**54.4**	**36.7**	**48.1**	**53.6**	**73.7**	**65.3**

4.3 Quantitative Results

We provide a detailed comparison of our models in terms of speed and accuracy with recent methods in Tab. 1. For efficiency, we compare total parameters, input resolution, number of input frames, and throughput. Overall, our Mobile-VideoGPT-0.5B variant performs comparably to LLaVA-OneVision [21] while being twice as fast (45.9 vs. 22.7 tokens/sec for LLaVA-OneVision-0.5B). Similarly, our 1.5B variant achieves results close to the recently released LLaVA-Mini-8B [65] while running up to 9× faster (41.0 vs. 4.6 tokens/sec for LLaVA-Mini-8B). These results demonstrate the advantages of our architectural design and training approach, enabling strong performance while maintaining efficiency. On ActivityNet-QA, our 0.5B model achieves 51.6% accuracy, outperforming LLaVA-OneVision-0.5B by 1.1%. Our 1.5B model achieves a score of 54.4%, surpassing LLaVA-Mini-8B by 2.1% and InternVL-2.5-2B by 0.3%, and coming just 0.9% behind the closed-source Gemini-1.5-Flash. Since ActivityNet-QA focuses on activity understanding in videos, the presence of the video encoder enables our model to capture rich temporal dependencies, leading to strong performance

on activity-centric benchmarks. Although our model is not explicitly designed for long videos, our models still achieve competitive results on the long-video MLVU benchmark, highlighting the effectiveness of our dual-encoder design. Notably, the recently introduced InternVL2.5-2B achieves marginally higher average performance than Mobile-VideoGPT-1.5B. However, our model uses 30% fewer parameters with 2× higher throughput.

Similarly, on MVBench, our 0.5B model achieves a score of 53.5, surpassing LLaVA-Mini-8B and LLaVA-OneVision-0.5B by 9% and 6.3%, respectively. MVBench evaluates video MLLMs across 20 temporal understanding sub-tasks, and we provide a detailed comparison of our 0.5B model against previous approaches for each sub-task in Tab. 2. Notably, our model performs well in tasks requiring strong temporal reasoning. For example, in the Moving Direction category, our 0.5B model scores 59.0%, significantly higher than the second-best score of 31.0% by LLaVA-Mini. Similar trends are observed in Action Localization, Action Count, Moving Count, Moving Attribute, and State Change sub-tasks. We attribute this strong performance to our attention-based frame selection algorithm coupled with the video encoder, which effectively captures video-specific dependencies. Notably, our model performs less effectively on fine-grained tasks, likely due to lower input resolution and the limited number of frames we use to maintain the efficiency of our design with real-time speed. A detailed discussion about the efficiency and performance on fine-grained tasks is provided in the Sec. 4.4

To evaluate open-ended generation, we use the VideoChatGPT benchmark [38]. Since the original setup relied on GPT-3.5-turbo-0613 (which is now deprecated), we re-evaluated both LLaVA-OneVision-0.5B and Mobile-VideoGPT-0.5B with GPT-3.5-turbo-1106, achieving average scores of 2.60 and 2.68, respectively, demonstrating that our lightweight model attains superior open-ended performance while being significantly more efficient.

4.4 Ablations

In Tab. 3, we present ablation studies evaluating our dual-encoder design in Mobile-VideoGPT, alongside the contributions of our frame-selection algorithm and token projection module. Across most benchmarks, the dual-encoder consistently outperforms single-encoder variants. The exception is the MLVU benchmark, where the video-encoder-only variant achieves the highest performance. This is because the video-only model processes all 16 frames, fully capturing temporal dynamics, whereas the dual-encoder leverages frame selection to choose 8 key frames complemented by image-level features, which slightly reduces temporal coverage. Notably, when frame selection (FS) is removed in the dual-encoder, performance on MLVU increases to 48.3%, surpassing the video-only variant, confirming that temporal coverage is critical for this benchmark. These findings provide two key insights. First, the dual-encoder design effectively captures complementary spatial and temporal cues, leading to robust performance across a wide range of tasks while maintaining efficiency. Second, for benchmarks or sce-

Table 2. Detailed performance on MVBench. Comparison across 20 sub-tasks grouped into nine categories against Video-ChatGPT, Video-LLaMA, Video-LLaVA, LLaVA-Mini, and LLaVA-OneVision. Best results are in **bold** and second-best are underlined. Despite its compact 0.5B scale, Mobile-VideoGPT achieves the highest overall average, and demonstrates particularly strong performance in counting, positional reasoning, attribute understanding, and object-centric tasks.

Spatial	Temporal	V-ChatGPT	V-LLaMA	V-LLaVA	LLaVA-M	LLaVA-OV	Ours
Average		32.7	34.1	43.1	44.5	47.2	**53.5**
Action	Action Sequence	23.5	27.5	44.5	44.5	**57.0**	54.0
	Action Prediction	26.0	25.5	50.0	44.5	**64.5**	56.5
	Action Antonym	62.0	51.0	49.0	**76.0**	49.5	56.0
	Fine-grained Act.	22.5	29.0	**42.0**	37.0	38.0	36.5
	Unexpected Act.	26.5	39.0	54.5	58.5	**70.0**	60.5
Object	Object Existence	54.0	48.0	52.5	50.0	53.5	**82.5**
	Object Interaction	28.0	40.5	46.5	50.0	**68.5**	51.0
	Object Shuffle	40.0	38.0	**40.5**	29.5	37.5	37.5
Position	Moving Direction	23.0	22.5	27.0	31.0	20.0	**59.0**
	Action Localiz.	20.0	22.5	28.5	32.5	27.0	**38.5**
Scene	Scene Trans.	31.0	43.0	84.5	85.5	**87.0**	82.0
Count	Action Count	30.5	34.0	44.5	35.0	40.5	**52.5**
	Moving Count	25.5	22.5	26.5	40.0	36.5	**63.5**
Attribute	Moving Attr.	39.5	32.5	53.0	48.0	54.5	**81.0**
Pose	State Change	48.5	45.5	38.5	41.0	40.0	**59.0**
	Fine-grained Pose	29.0	32.5	34.0	29.5	**43.0**	28.0
Character	Char. Order	33.0	40.0	42.5	52.0	**55.5**	47.0
Cognition	Ego. Navigation	29.5	30.0	**32.5**	31.0	26.5	31.5
	Episodic Reas.	26.0	21.0	38.0	38.0	**38.5**	37.0
	Counterf. Inf.	35.5	37.0	32.0	36.0	35.5	**57.5**

Table 3. Systematic ablation study of architectural components. We evaluate the impact of vision encoder types, frame selection (FS) strategies, and token projection methods on performance and throughput. Highlights in color indicate our final design choices.

Benchmark	Vision Encoder			Frame Selection (FS)		Token Projection	
	Image	Video	Dual w/ FS	w/o FS	Attn-Based	MLP_{proj}	ET_{proj}
ActNet-QA	51.4	49.3	**51.6**	51.3	**51.6**	50.9	**51.6**
EgoSchema	26.5	27.1	**31.4**	31.2	**31.4**	30.5	**31.4**
MLVU	43.0	**48.2**	47.9	**48.3**	47.9	45.0	**47.9**
MVBench	52.4	49.6	**53.5**	**54.0**	53.5	**54.9**	53.5
NextQA	62.1	62.6	**65.4**	65.3	**65.4**	64.9	**65.4**
PercepTest	58.3	58.0	**58.7**	**58.7**	**58.7**	**59.5**	58.7
Avg. Accuracy	49.0	49.1	**51.4**	**51.5**	51.4	51.0	**51.4**
Throughput	49.6	41.3	**45.9**	38.5	**45.9**	39.2	**45.9**

narios requiring dense temporal information, such as MLVU, there is a trade-off between efficiency and temporal completeness.

Table 4. Ablation on frame configurations and efficiency. We compare fine-grained action and pose recognition performance against inference throughput. FS (Frame Selection) improves efficiency (Throughput) but introduces a slight trade-off in accuracy. Best results are in **bold**.

Setting	Fine-grained Action ↑	Fine-grained Pose ↑	Throughput (fps) ↑
16 frames	39.8	32.4	38.5
16 frames + FS	36.5	28.0	**45.9**
32 frames	**43.3**	**41.6**	30.2
32 frames + FS	41.2	37.8	36.7

Although the dual-encoder has slightly lower throughput (45.9 vs. 49.6 tokens/sec) than the image-only variant, it achieves around 5% higher accuracy on challenging tasks like EgoSchema and MLVU. Our lightweight video encoder (VideoMamba with 73M parameters) adds minimal overhead, making this an efficient trade-off for real-time edge deployment with enhanced accuracy. Overall, the ablation study demonstrates that the dual-encoder design offers a favorable balance between efficiency and spatiotemporal representation, highlighting its potential for scalable, long-video understanding while maintaining generalizability across video benchmarks. To evaluate the effectiveness of our frame-selection method, we train the model without frame selection, meaning the video encoder processes all video frames. The results show that our attention-based frame-selection performs on-bar by processing all video frames while achieving higher throughput. Finally, we compare our efficient token projector against the widely used MLP-based projection, showing improved accuracy while maintaining higher throughput.

While our model demonstrates strong overall performance, its effectiveness on fine-grained tasks such as action and pose recognition is relatively limited. We attribute this to two primary design choices aimed at maximizing efficiency and enabling real-time inference: (i) using a lower spatial resolution compared to larger baselines, and (ii) constraining the number of frames to reduce temporal redundancy. Both factors, while beneficial for speed and memory efficiency, may discard subtle spatial and temporal cues that are critical for distinguishing fine-grained categories.

To better understand this trade-off, we conducted additional ablation by varying the number of frames with our frame selection strategies. Tab. 4 reports the results on fine-grained action and pose tasks. We observe that increasing the frame count from 16 to 32 improves performance because the model sees more informative temporal segments. However, this comes at the cost of higher computational overhead with lower throughput. For instance, increasing the number of input frames from 16 to 32 improves the fine-grained action by 3.7% and fine-grained pose by 9.2%, while reducing the throughput from 38.5 to 30.2. These findings highlight the fundamental tension between efficiency and fine-grained recognition.

Fig. 4. Qualitative comparison of Mobile-VideoGPT-0.5B against LLaVA-OneVision-0.5B and LLaVA-Mini-8B. Mobile-VideoGPT produces more accurate and contextually grounded responses across diverse video understanding scenarios, including temporal reasoning, action recognition, and scene comprehension. Beyond response quality, our model achieves superior efficiency, with lower latency and higher throughput (tokens per second), demonstrating that compact design need not sacrifice comprehension quality. More qualitative examples are presented in the suppl. material.

4.5 Qualitative Results

Fig. 4 compares Mobile-VideoGPT with two recent baselines (LLaVa-Mini-8B [65] and LLaVa-OneVision-0.5B [21]) on two video segments: a nighttime field hockey game (top) and a skateboarding session (bottom). The baselines provide incomplete or generic descriptions, missing key details such as the final score in the hockey game. In contrast, Mobile-VideoGPT captures the full temporal progres-

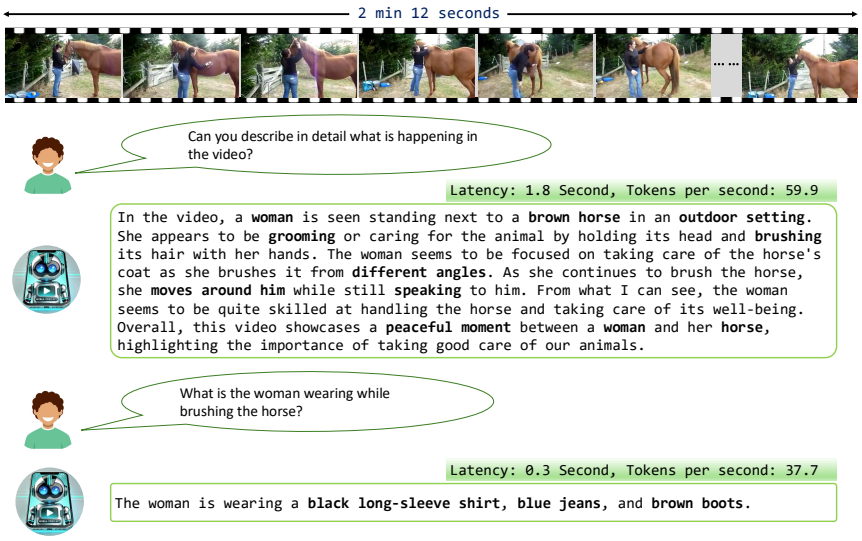


Fig. 5. Qualitative results of Mobile-VideoGPT-0.5B on open-ended video question answering. Our model demonstrates strong video comprehension capabilities across real-world scenarios, accurately responding to both short factual and long descriptive open-ended questions. Mobile-VideoGPT produces detailed, rich, temporally coherent answers while maintaining real-time inference throughput (tokens per second), underscoring its suitability for efficient on-device video understanding.

sion with detailed outputs, while also achieving over 40 tokens per second with lower latency. Mobile-VideoGPT-0.5B is 2–3× faster than LLaVa-OneVision-0.5B and 5× faster than LLaVa-Mini in the top video, and over 10× faster in the bottom video.

We show in Fig. 5 a representative example in which our Mobile-VideoGPT-0.5B model responds to both detailed and concise queries regarding a short video clip. When asked to describe the scene, the model accurately explains that the woman is brushing and caring for the horse, highlighting key actions and the surrounding setting. It then handles a more focused query—what the woman is wearing—by providing a concise and correct answer, showcasing its ability to manage multi-turn conversations of varying complexity within a single inference session. Beyond response accuracy, the model demonstrates strong temporal grounding, correctly attributing actions to the appropriate visual context without hallucination or temporal drift. Notably, the model maintains low latency (1.8 seconds for the detailed description and 0.3 seconds for the concise follow-up) and a high token throughput, making it well-suited for real-time, on-device video understanding tasks across a range of query types.

5 Conclusion

We tackle the computational inefficiency in video understanding through an efficient multimodal architecture designed for real-time deployment on edge devices. By combining lightweight visual encoders, efficient token projectors, and small language models, our framework reduces the computational overhead while maintaining strong performance. The innovative Attention-Based Frame-Scoring dynamically selects key-frames, while efficient token projection retains critical visual context, enabling real-time throughput. Our evaluation across six benchmarks shows that our model achieves competitive performance, while being 2× faster and 3× smaller compared to Llava-One-Vision-0.5B on Jetson Orin Nano, which highlights the model's strong practicality on edge devices.

References

1. AI, M.: Llama 3. `https://llama.meta.com/llama3` (2024)
2. Ataallah, K., et al.: Goldfish: Vision-language understanding of arbitrarily long videos. ECCV (2024)
3. Bain, M., Nagrani, A., Varol, G., Zisserman, A.: Frozen in time: A joint video and image encoder for end-to-end retrieval. In: ICCV (2021)
4. Buch, S., Nagrani, A., Arnab, A., Schmid, C.: Flexible frame selection for efficient video reasoning. In: CVPR (2025)
5. Chen, Z., Wang, W., Cao, Y., Liu, Y., Gao, Z., Cui, E., Zhu, J., Ye, S., Tian, H., Liu, Z., et al.: Expanding performance boundaries of open-source multimodal models with model, data, and test-time scaling. arXiv preprint arXiv:2412.05271 (2024)
6. Cheng, Z., et al.: Videollama 2: Advancing spatial-temporal modeling and audio understanding in video-llms. arXiv preprint arXiv:2406.07476 (2024)
7. Chu, X., et al.: Mobilevlm: A fast, reproducible and strong vision language assistant for mobile devices. arXiv preprint arXiv:2312.16886 (2023)
8. Chung, J., Zhu, T., Saez-Diez, M.G., Niebles, J.C., Zhou, H., Russakovsky, O.: Unifying specialized visual encoders for video language models. arXiv preprint arXiv:2501.01426 (2025)
9. Frantar, E., Alistarh, D.: SparseGPT: Massive language models can be accurately pruned in one-shot. arXiv preprint arXiv:2301.00774 (2023)
10. Gemini: Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context (2024)
11. Gemini, et al.: Gemini: A family of highly capable multimodal models. arXiv preprint arXiv:2312.11805 (2023)
12. Goyal, R., et al.: The" something something" video database for learning and evaluating visual common sense. In: ICCV (2017)
13. Grauman, K., et al.: Ego4d: Around the world in 3,000 hours of egocentric video. In: CVPR (2022)
14. Han, S., Mao, H., Dally, W.J.: Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In: ICLR (2016)
15. He, Y., Zhang, X., Sun, J.: Channel pruning for accelerating very deep neural networks. In: ICCV. pp. 1389–1397 (2017)
16. Hinton, G., Vinyals, O., Dean, J.: Distilling the knowledge in a neural network. In: NIPS Workshop on Deep Learning and Representation Learning (2015)

17. Houlsby, N., et al.: Parameter-efficient transfer learning for nlp. In: ICLR (2019)
18. Hu, E.J., et al.: LoRA: Low-rank adaptation of large language models. In: ICML (2022)
19. Huang, X., Zhou, H., Han, K.: Prunevid: Visual token pruning for efficient video large language models. In: arXiv (2024)
20. Kay, W., Carreira, J., Simonyan, K., Zhang, B., Hillier, C., Vijayanarasimhan, S., Viola, F., Green, T., Back, T., Natsev, P., et al.: The kinetics human action video dataset. arXiv preprint arXiv:1705.06950 (2017)
21. Li, B., et al.: Llava-onevision: Easy visual task transfer. arXiv preprint arXiv:2408.03326 (2024)
22. Li, J., Li, D., Savarese, S., Hoi, S.: BLIP-2: bootstrapping language-image pre-training with frozen image encoders and large language models. In: ICML (2023)
23. Li, K., He, Y., Wang, Y., Li, Y., Wang, W., Luo, P., Wang, Y., Wang, L., Qiao, Y.: Videochat: Chat-centric video understanding. arXiv preprint arXiv:2305.06355 (2023)
24. Li, K., Li, X., Wang, Y., He, Y., Wang, Y., Wang, L., Qiao, Y.: Videomamba: State space model for efficient video understanding. In: ECCV (2024)
25. Li, K., Li, X., Wang, Y., He, Y., Wang, Y., Wang, L., Qiao, Y.: Videomamba: State space model for efficient video understanding (2024)
26. Li, K., Wang, Y., He, Y., Li, Y., Wang, Y., Liu, Y., Wang, Z., Xu, J., Chen, G., Luo, P., et al.: Mvbench: A comprehensive multi-modal video understanding benchmark. In: CVPR (2024)
27. Li, L., Li, Q., Zhang, B., Chu, X.: Norm tweaking: High-performance low-bit quantization of large language models. In: AAAI (2024)
28. Li, W., Yuan, Y., Liu, J., Tang, D., Wang, S., Zhu, J., Zhang, L.: Tokenpacker: Efficient visual projector for multimodal llm (2024)
29. Li, Y., Wang, C., Jia, J.: Llama-vid: An image is worth 2 tokens in large language models. In: ECCV (2024)
30. Li, Y., Bubeck, S., Eldan, R., Del Giorno, A., Gunasekar, S., Lee, Y.T.: Textbooks are all you need ii: phi-1.5 technical report. arXiv preprint arXiv:2309.05463 (2023)
31. Liang, H., et al.: Keyvideollm: Towards large-scale video keyframe selection. arXiv preprint arXiv:2407.03104 (2024)
32. Lin, B., Zhu, B., Ye, Y., Ning, M., Jin, P., Yuan, L.: Video-llava: Learning united visual representation by alignment before projection. arXiv preprint arXiv:2311.10122 (2023)
33. Lin, B., Zhu, B., Ye, Y., Ning, M., Jin, P., Yuan, L.: Video-llava: Learning united visual representation by alignment before projection. arXiv preprint arXiv:2311.10122 (2023)
34. Liu, A., et al.: Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437 (2024)
35. Liu, H., Li, C., Li, Y., Lee, Y.J.: Improved baselines with visual instruction tuning. arXiv preprint arXiv:2310.03744 (2023)
36. Liu, H., Li, C., Wu, Q., Lee, Y.J.: Visual instruction tuning. In: NIPS (2023)
37. Liu, Z., et al.: Mobilellm: Optimizing sub-billion parameter language models for on-device use cases. arXiv preprint arXiv:2402.14905 (2024)
38. Maaz, M., Rasheed, H., Khan, S., Khan, F.S.: Video-chatgpt: Towards detailed video understanding via large vision and language models (2024)
39. Maaz, M., Rasheed, H., Khan, S., Khan, F.S.: Videogpt+: Integrating image and video encoders for enhanced video understanding. arxiv (2024)

40. Maaz, M., et al.: Edgenext: Efficiently amalgamated cnn-transformer architecture for mobile vision applications. In: International Workshop on Computational Aspects of Deep Learning at 17th European Conference on Computer Vision (CADL2022). Springer (2022)
41. Mangalam, K., Akshulakov, R., Malik, J.: Egoschema: A diagnostic benchmark for very long-form video language understanding. NIPS (2023)
42. NVIDIA Corporation: Jetson Orin Series: AI Performance for Edge Devices (2022), https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/
43. OpenAI: Gpt-4v. https://openai.com/index/gpt-4v-system-card/ (2023)
44. OpenAI: Gpt-4v(ision) system card. https://api.semanticscholar.org/CorpusID:263218031 (2023)
45. OpenAI: Hello gpt-4o. https://openai.com/index/hello-gpt-4o/ (2024)
46. Pătrăucean, V., et al.: Perception test: A diagnostic benchmark for multimodal video models. arXiv preprint arXiv:2305.13786 (2023)
47. Peng, Z., Wang, W., Dong, L., Hao, Y., Huang, S., Ma, S., Wei, F.: Kosmos-2: Grounding multimodal large language models to the world. ArXiv **abs/2306** (2023)
48. Radford, A., et al.: Learning transferable visual models from natural language supervision. In: ICML (2021)
49. Rasheed, H., et al.: Glamm: Pixel grounding large multimodal model. The IEEE/CVF Conference on Computer Vision and Pattern Recognition (2024)
50. Rasheed, H., et al.: Palo: A large multilingual multimodal language model. In: WACV (2025)
51. Romero, A., Ballas, N., Kahou, S.E., Chassang, A., Gatta, C., Bengio, Y.: Fitnets: Hints for thin deep nets. In: ICLR (2015)
52. Shaker, A., et al.: Swiftformer: Efficient additive attention for transformer-based real-time mobile vision applications. In: ICCV (2023)
53. Tang, X., Qiu, J., Xie, L., Tian, Y., Jiao, J., Ye, Q.: Adaptive keyframe sampling for long video understanding. In: CVPR (2025)
54. Touvron, H., et al.: Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023)
55. Touvron, H., et al.: Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971 (2023)
56. Wang, K., Xuan, H.: Llava-zip: Adaptive visual token compression with intrinsic image information (2024), https://arxiv.org/abs/2412.08771
57. Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., Han, S.: Smoothquant: Accurate and efficient post-training quantization for large language models. In: ICML (2023)
58. Xiao, J., Shang, X., Yao, A., Chua, T.S.: Next-qa: Next phase of question-answering to explaining temporal actions. In: CVPR (2021)
59. Yang, A., et al.: Qwen2 technical report. arXiv preprint arXiv:2407.10671 (2024)
60. Yao, Z., Wu, X., Li, C., Youn, S., He, Y.: Exploring post-training quantization in llms from comprehensive study to low rank compensation. In: AAAI (2024)
61. Yi, K., Gan, C., Li, Y., Kohli, P., Wu, J., Torralba, A., Tenenbaum, J.B.: Clevrer: Collision events for video representation and reasoning. arXiv preprint arXiv:1910.01442 (2019)
62. Yu, Z., othres: Activitynet-qa: A dataset for understanding complex web videos via question answering. In: AAAI (2019)
63. Zhang, P., Zeng, G., Wang, T., Lu, W.: Tinyllama: An open-source small language model. arXiv preprint arXiv:2401.02385 (2024)

64. Zhang, P., et al.: Long context transfer from language to vision. arXiv preprint arXiv:2406.16852 (2024)
65. Zhang, S., Fang, Q., Yang, Z., Feng, Y.: Llava-mini: Efficient image and video large multimodal models with one vision token. arXiv preprint arXiv:2501.03895 (2025)
66. Zhang, Y., Wu, J., Li, W., Li, B., Ma, Z., Liu, Z., Li, C.: Video instruction tuning with synthetic data (2024)
67. Zhou, J., et al.: Mlvu: A comprehensive benchmark for multi-task long video understanding. arXiv preprint arXiv:2406.04264 (2024)
68. Zhu, D., Chen, J., Shen, X., Li, X., Elhoseiny, M.: Minigpt-4: Enhancing vision-language understanding with advanced large language models. In: ICLR (2024)
69. Zohar, O., et al.: Apollo: An exploration of video understanding in large multimodal models. arXiv preprint arXiv:2412.10360 (2024)

6 Supplementary Material

In this section, we provide more details regarding the following:-

- Discussion about Fine-Grained Tasks vs. Efficiency
- Additional Benchmark Details
- Additional Training Details
- Comparison between the visual tokens for Mobile-VideoGPT and LLaVa-OneVision
- Additional Qualitative Results

6.1 Discussion about Fine-Grained Tasks vs. Efficiency

As discussed in the main paper, the limited fine-grained recognition of Mobile-VideoGPT stems from two efficiency-driven design choices: lower spatial resolution and a constrained frame count. Tab. 4 further examines this trade-off by varying the number of frames alongside our frame selection (FS) strategy. Increasing the frame count from 16 to 32 yields consistent gains in fine-grained action (+3.7%) and pose (+9.2%) recognition by capturing more informative temporal segments, albeit at the cost of reduced throughput (38.5→30.2 fps). Applying FS partially recovers this efficiency gap (*e.g.*, 32 frames + FS reaches 36.7 fps) with only a modest accuracy drop, highlighting its role as an effective efficiency–accuracy balancer.

These findings underscore the fundamental tension between efficiency and fine-grained recognition. A promising direction for future work is to incorporate adaptive mechanisms that dynamically allocate higher spatial or temporal resolution only when fine-grained distinctions are required, while maintaining efficiency for simpler cases.

6.2 Additional benchmark details

ActivityNet-QA [62] is a short question-answering dataset derived from the popular ActivityNet [62] dataset. The test set consists of 8K samples. Following Video-ChatGPT [38], we employ GPT-3.5-Turbo for evaluation.

EgoSchema [41] is an MCQ-based benchmark where the videos are sourced from Ego4D [13]. The test set consists of 5K samples, and the results are compiled by submitting the model's predictions to the online evaluation server.

MLVU [67] is a multimodal long video understanding benchmark encompassing nine distinct tasks, including MCQs as well as open-ended generation tasks. Following LLaVA-OneVision [21], we report results on the dev set, which consists of 2,593 samples in total. Although our model is not specifically designed for long video understanding, it achieves competitive performance.

MVBench [26] is a comprehensive multimodal video understanding benchmark containing 20 sub-tasks, including Action Sequence (AS), Action Prediction (AP), Action Antonym (AA), Fine-grained Action (FA), Unexpected Action (UA), Object Existence (OE), Object Interaction (OI), Object Shuffle (OS),

Table 5. Training configuration of Mobile-VideoGPT across three stages. Stage 1 pre-trains the image projection layer while keeping all other modules frozen. Stage 2 extends pre-training to the video projection layer. Stage 3 performs full instruction tuning with the language model unfrozen. All stages use AdamW with cosine decay scheduling.

Settings		Stage 1 *Image Proj. Pre-training*	Stage 2 *Video Proj. Pre-training*	Stage 3 *Instruction Tuning*
Modules	Image Encoder	Frozen	Frozen	Frozen
	Video Encoder	Frozen	Frozen	Frozen
	Small Language Model	Frozen	Frozen	Trainable
	Projection	Trainable	Trainable	Trainable
Hyperparameters	Batch Size	128	128	64
	Learning Rate	1e-3	1e-3	2e-4
	Schedule	Cosine decay	Cosine decay	Cosine decay
	Warmup Ratio	0.03	0.03	0.03
	Optimizer	AdamW	AdamW	AdamW
	Epochs	2	2	2

Moving Direction (MD), Action Localization (AL), Scene Transition (ST), Action Count (AC), Moving Count (MC), Moving Attribute (MA), State Change (SC), Fine-grained Pose (FP), Character Order (CO), Egocentric Navigation (EN), Episodic Reasoning (ER), and Counterfactual Inference (CI). Each subtask includes 200 MCQs, totaling 4K MCQs in the benchmark.

NExTQA [58] is a video question-answering benchmark that focuses on explaining temporal actions in videos. Following previous approaches, we evaluate on the MCQ test set consisting of 8,564 samples, targeting causal action reasoning, temporal action reasoning, and common scene comprehension.

PerceptionTest [46] is designed to assess the perception and reasoning capabilities of multimodal models. Following previous approaches, we report results on the validation set, which consists of 19,140 samples.

6.3 Additional training details

We adopt a three-stage training pipeline as summarized in Tab. 5. In **Stage 1 (Image Projector Pre-training)**, the image and video encoders, as well as the small language model, are kept frozen. Only the efficient image token projection module is trainable. We use a batch size of 128, a learning rate of 1×10^{-3}, and a cosine decay schedule with a warmup ratio of 0.03 for two epochs, optimized by AdamW. **Stage 2 (Video Projector Pre-training)** follows the same hyperparameter settings, again freezing the image/video encoders and the small language model, and training only the projection module. **Stage 3 (Instruction Tuning)** keeps the encoders frozen but makes both the projection module and the small language model trainable. The latter is trained with LoRA, configured as $lora_r = 128$ and $lora_\alpha = 256$. In this stage, we use a smaller learning rate of 2×10^{-4}, a batch size of 64, and maintain the same optimizer and schedule strategy for two additional epochs. Pre-training of the image and video projectors takes approximately 16 hours on 8 A100 40 GB GPUs, while the instruction fine-tuning stage requires around 36 hours.

Table 6. Token efficiency comparison between LLaVA-OneVision and Mobile-VideoGPT. Mobile-VideoGPT uses efficient token projection (ET_{Proj}) combined with frame sampling (FS), reducing total visual tokens from 6,272 to 2,696, a **2.3× reduction**, while separately handling image and video tokens through dedicated projection heads.

Model	Projection Strategy	Image Tokens	Video Tokens	Total Tokens
LLaVA-OneVision	MLP_{Proj}	$32 \times 196 = 6{,}272$	–	6,272
Mobile-VideoGPT (Ours)	ET_{Proj} + FS	$16 \times 144 = 2{,}304$	$8 \times 49 = 392$	2,696

6.4 Comparison between the visual tokens for Mobile-VideoGPT and LLaVa-OneVision

Tab. 6 compares Mobile-VideoGPT against LLaVA-OneVision in terms of projection strategies and their resulting token counts. LLaVA-OneVision uses a simple MLP-based projector (MLP_{Proj}) that handles 32 input frames, each producing 196 tokens, for a total of 6272 tokens. By contrast, our method relies on an Efficient Token Projection layer (ET_{Proj}) combined with frame selection (FS). For images, we consider 16 frames, each represented by 144 tokens, yielding 2304 tokens. For video, we select 8 of those frames, each producing 49 tokens, resulting an additional 392 tokens. Altogether, we generate a total of 2696 visual tokens, substantially reducing the visual token load while simultaneously extending the model's capabilities to video data, thereby underscoring the efficiency and flexibility of Mobile-VideoGPT.

6.5 Additional qualitative results

We show in Fig 6 additional representative example in which our Mobile-VideoGPT 0.5B model responds to both detailed and concise queries regarding a 3-minute video clip. When asked to describe the scene, the model accurately identifies a young boy playing the violin accompanied by an adult pianist, correctly noting details such as the boy's attire, their synchronized performance, and the warm setting. It then handles more focused follow-up queries—what the boy is wearing and why the adult is playing along—providing concise and correct answers, showcasing its ability to handle multi-turn conversations of varying complexity. Notably, the model maintains consistently low latency (0.2–0.3 seconds per response) and high token throughput (35–45 tokens per second), making it practical for real-time video understanding tasks on resource-constrained devices.

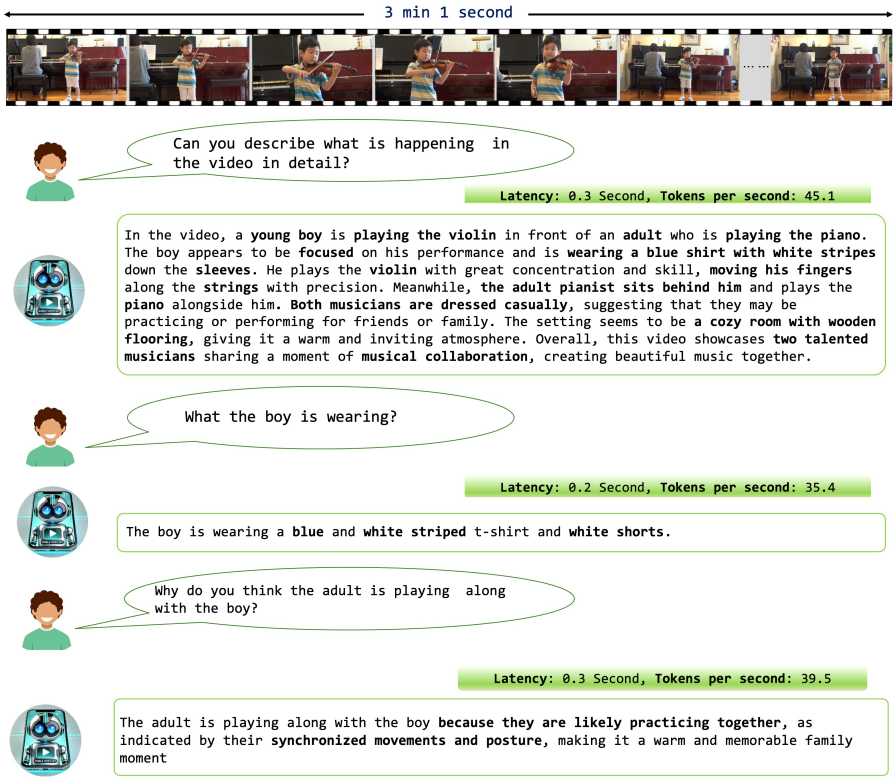


Fig. 6. Qualitative results of Mobile-VideoGPT-0.5B on open-ended video question answering. Our model demonstrates strong video comprehension capabilities across real-world scenarios, accurately responding to both short factual and long descriptive open-ended questions. Mobile-VideoGPT produces detailed, rich, temporally coherent answers while maintaining real-time inference throughput (tokens per second), underscoring its suitability for efficient on-device video understanding.